DIGITAL CLOCK

Progress log

-Sree Lasya, Bhavajna

Basic Idea

A digital clock made using an Arduino Uno, 7447 IC, and seven-segment displays works by having the Arduino keep track of the time and send each digit of the time to the 7447 IC in BCD form. The 7447 converts the BCD input into the required signals for the seven-segment display, which then lights the appropriate segments to show the numbers. For example, if the Arduino sends the BCD value for 5, the 7447 activates the segments needed to display 5. By continuously updating the digits for hours, minutes, and seconds, the system displays the current time.

Learning how the Arduino works and understanding components such as the 7447 decoder was especially interesting while building our digital clock. We learned how the Arduino controls the timing and sends signals to the different components, while the 7447 decoder converts the signals into the appropriate outputs for the seven-segment displays. Connecting the Arduino, 7447 ICs, displays, and other components helped us understand how a digital clock keeps track of seconds, minutes, and hours.

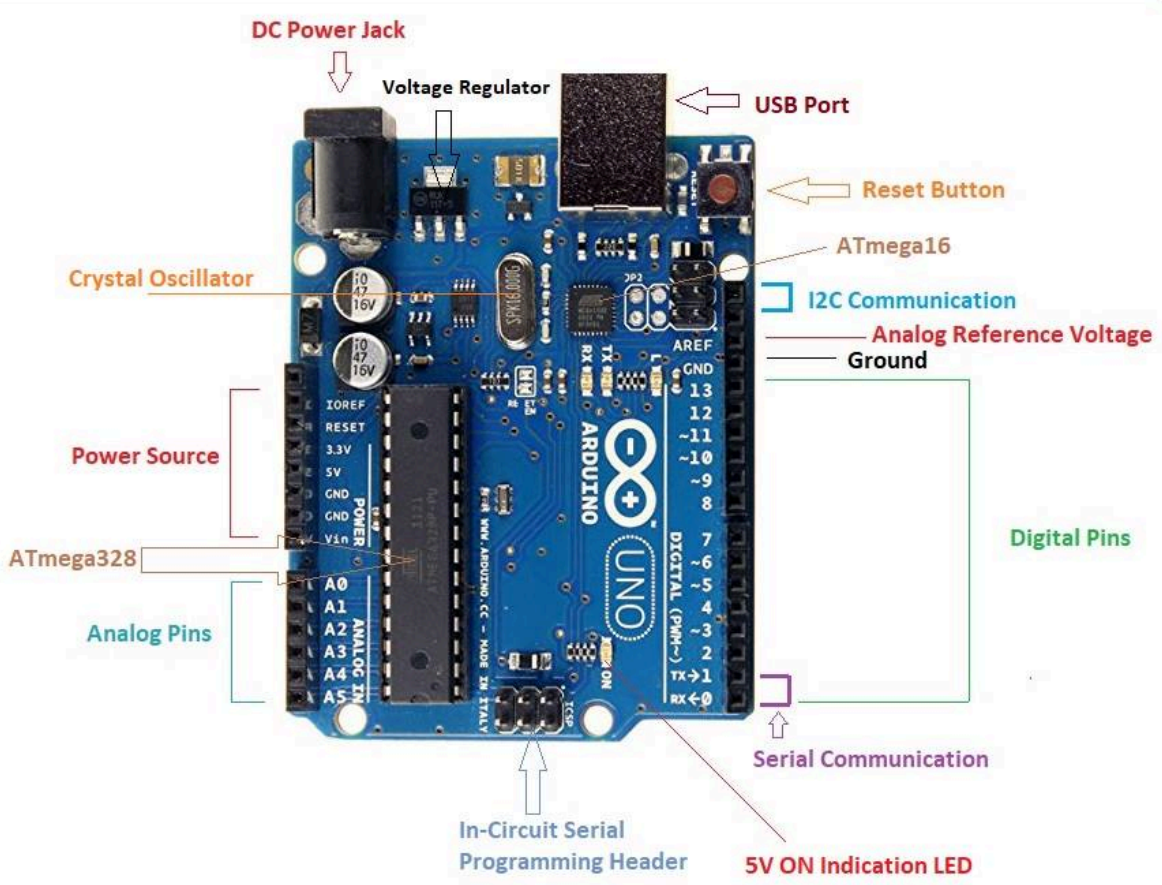
What is BCD?

BCD (Binary-Coded Decimal) is a way of representing each decimal digit using 4 binary bits (0s and 1s).

So, if the Arduino wants to display 5, it sends 0101 as the BCD input to the 7447 IC. The 7447 then converts this input into the correct signals to light up the seven-segment display and show **5**.(The segments that are on are: a — top, b — upper-left, g — middle, c — lower-right, d — bottom)

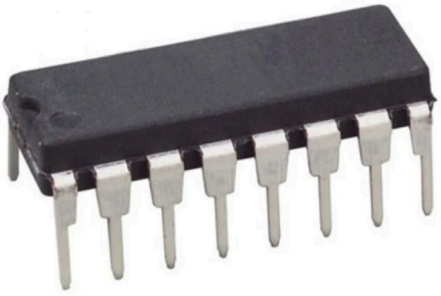
Components

1.Arduino UNO



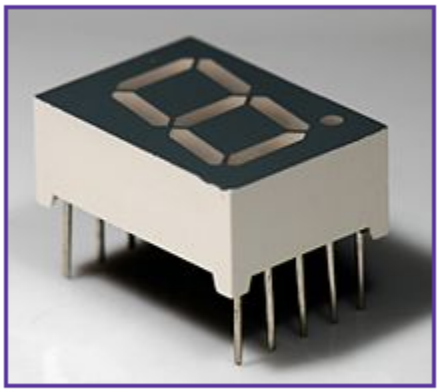
The Arduino Uno acts as the brain of the digital clock. In this project, it runs the programmed code to keep track of the hours, minutes, and seconds, processes the timing logic, and sends the required signals to control the display. The code continuously updates the time, allowing the clock to function automatically.

2. 7447



The 7447 IC is a BCD-to-seven-segment decoder that converts the 4-bit BCD input from the Arduino into signals for the seven-segment display. It determines which segments should turn ON or OFF to display the required number, such as 5. This reduces the number of control signals needed from the Arduino.

3. 7 segment display



A seven-segment display is an electronic display made of seven LED segments labeled a to g. By turning different segments ON or OFF, it displays numbers from 0 to 9. In the digital clock, it is used to display the hours, minutes, and seconds.

4.Other Components

Jumper wires,
Push buttons,
Resistors,
Usb cable,

MULTIPLEXING

Multiplexing is a method of controlling multiple displays using fewer connections. It allows several seven-segment displays to share the same data lines. The Arduino selects which display should be active and sends the required digit to the 7447 IC. The 7447 converts the BCD value into segment signals. Only one display is activated at a particular moment, and the Arduino quickly switches between the displays. Each display shows its required digit for a short time. Due to this rapid switching, all the digits appear continuously lit to the human eye. Thus, multiplexing makes the digital clock circuit simpler and reduces the amount of wiring and connections required.

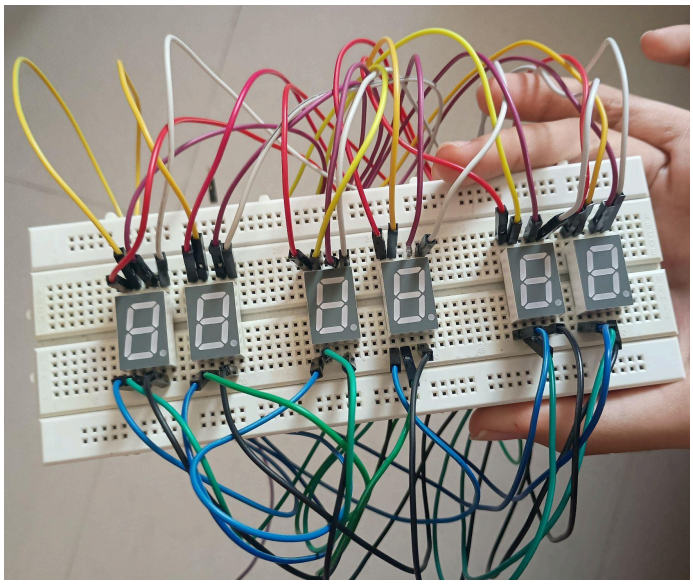
BUILDING THE PROJECT

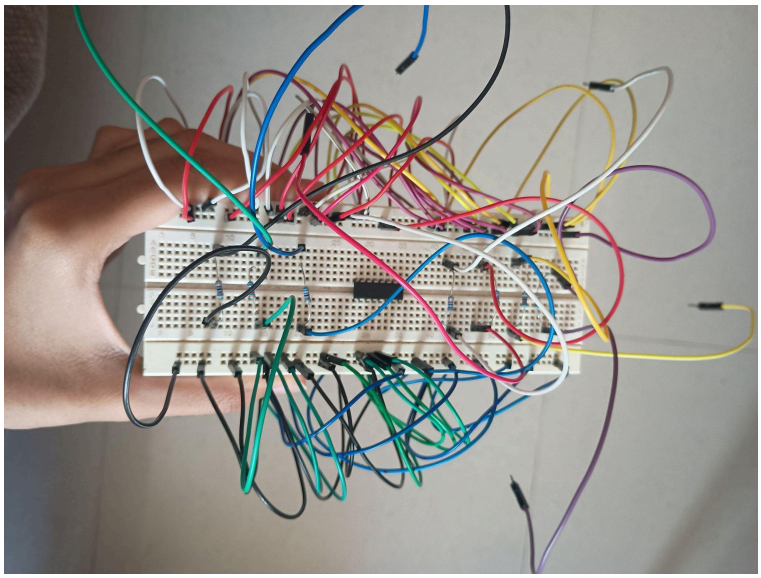
WIRING:

1. Connecting all segments

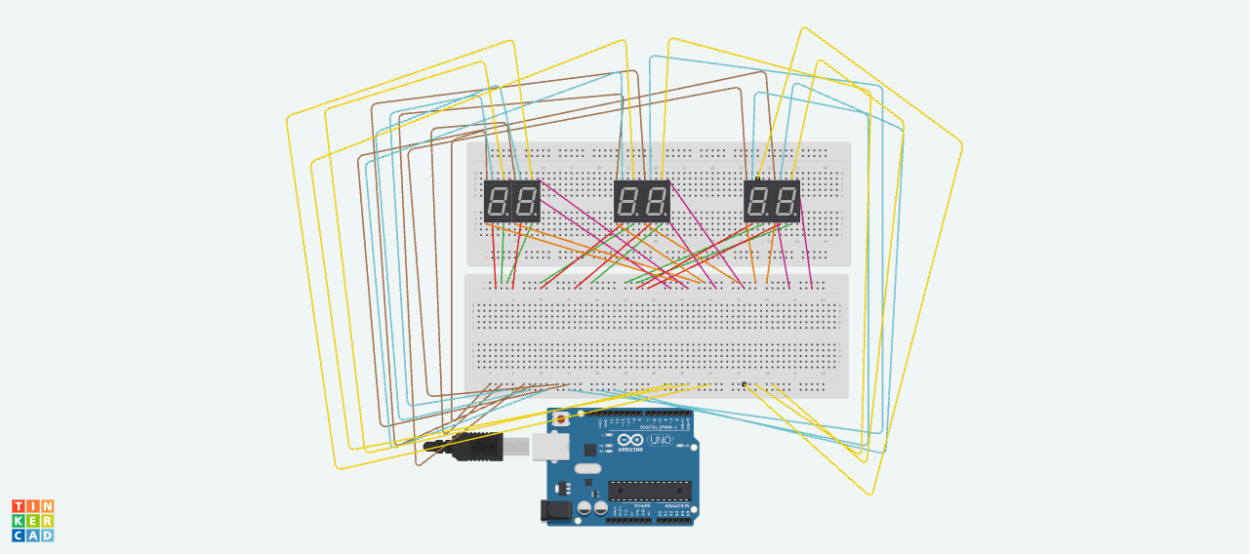
Bussing:

Bussing is the process of connecting multiple components in a circuit using common electrical lines called buses. In a digital clock, bussing helps connect the Arduino, 7447 IC, and seven-segment displays in an organized way. Common connections such as VCC, GND, and data lines can be shared between components.



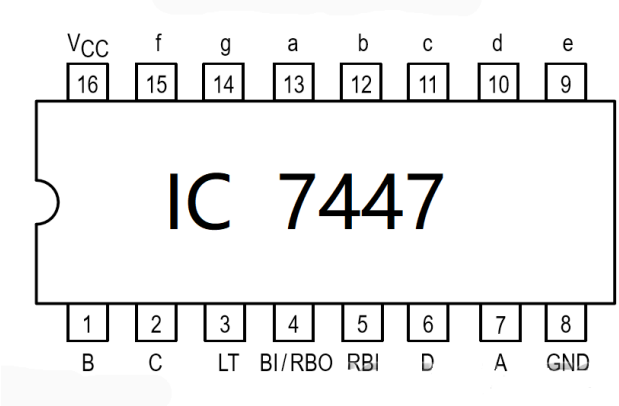


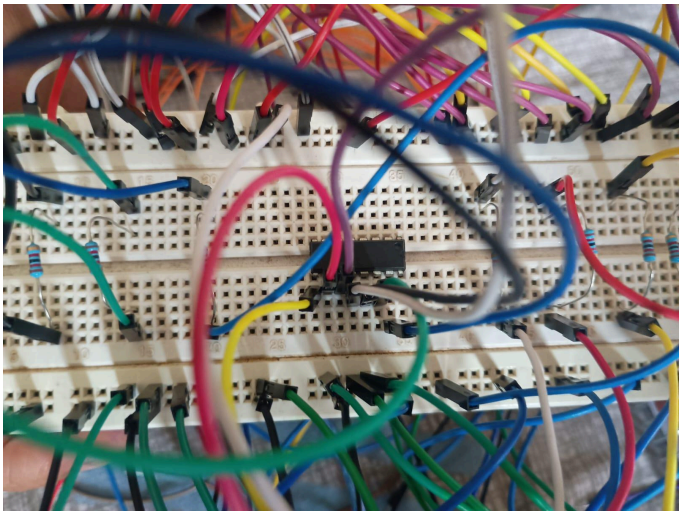
After placing the seven-segment displays and the 7447 IC on the breadboard, we bussed the a–g segment pins of all the seven-segment displays. Each corresponding segment pin was connected to the same bus line, creating a parallel connection between the displays. This allows the same segment signals from the 7447 to be shared across all the displays during multiplexing.



Tinkercad simulation

2.Connection to 7447





We then took a jumper wire for each of the segment bus and connected a resistor to it. Through the resistor the connection of each of the segment bus to 7447 has been established. The 7447 output connections are

7 segment display→ pin of 7447

a → pin 13,

b → pin 12,

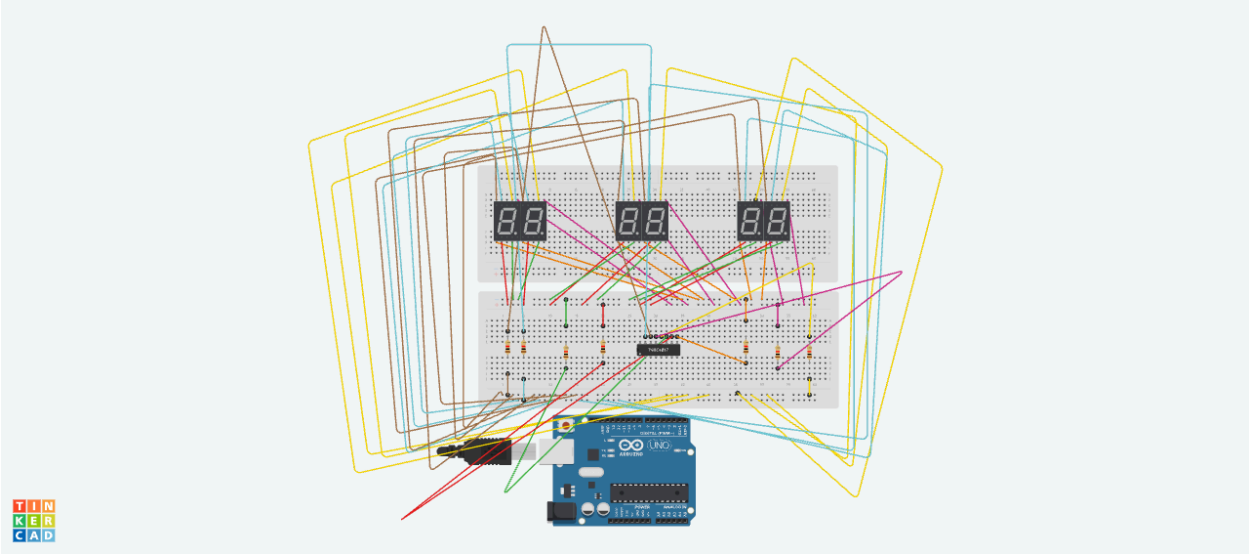
c → pin 11,

d → pin 10,

e → pin 9,

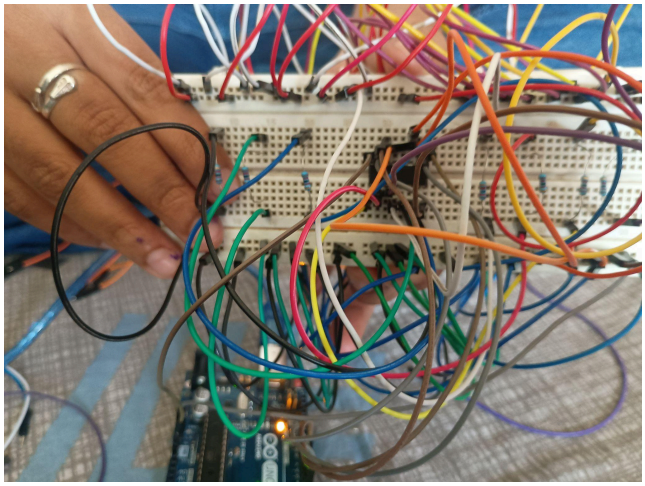
f → pin 15,

g → pin 14. The common-anode display is hence connected to the corresponding output pins of the 7447.



Tinkercad simulation

3. Arduino to 7447

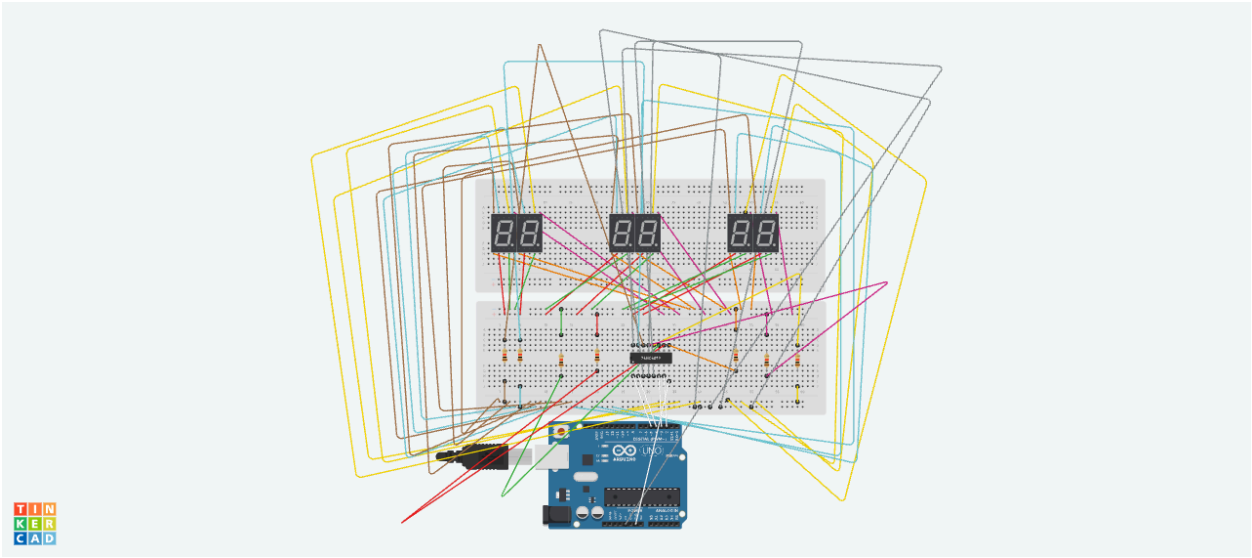


For an Arduino → 7447 connection, the BCD input pins are:

- Arduino D2 → 7447 pin 7 (A)
- Arduino D3 → 7447 pin 1 (B)
- Arduino D4 → 7447 pin 2 (C)
- Arduino D5 → 7447 pin 6 (D)

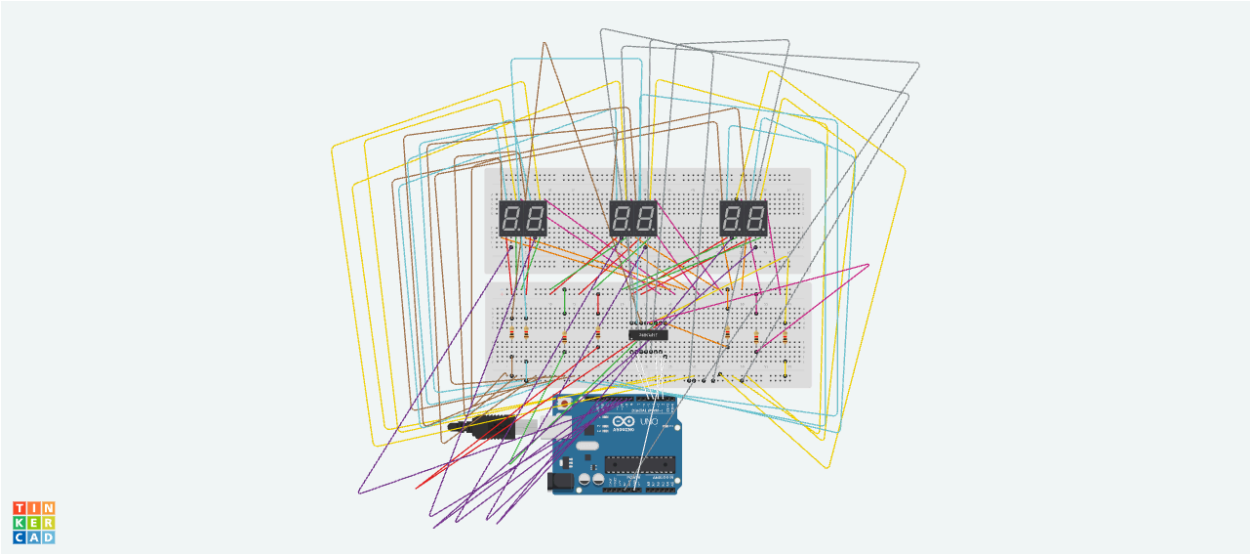
(We have connected 5V in arduino to the breadboard and connected 7447 pins that go to 5V to that common rail.)

- Connect the 7447 VCC (pin 16) ,pin 3,pin 4,pin 5→ +5V and GND (pin 8) → GND.



4. COM→Arduino connection

Hours tens→pin 8,
Hours units→pin 9,
Minutes tens→pin 10,
Minutes units→pin 11,
Secs tens→pin 12,
Secs units→pin 13,
Connect the 7447 VCC (pin 16) → +5V and GND (pin 8) → GND.
Arduino GND and 7447 GND must be connected together.

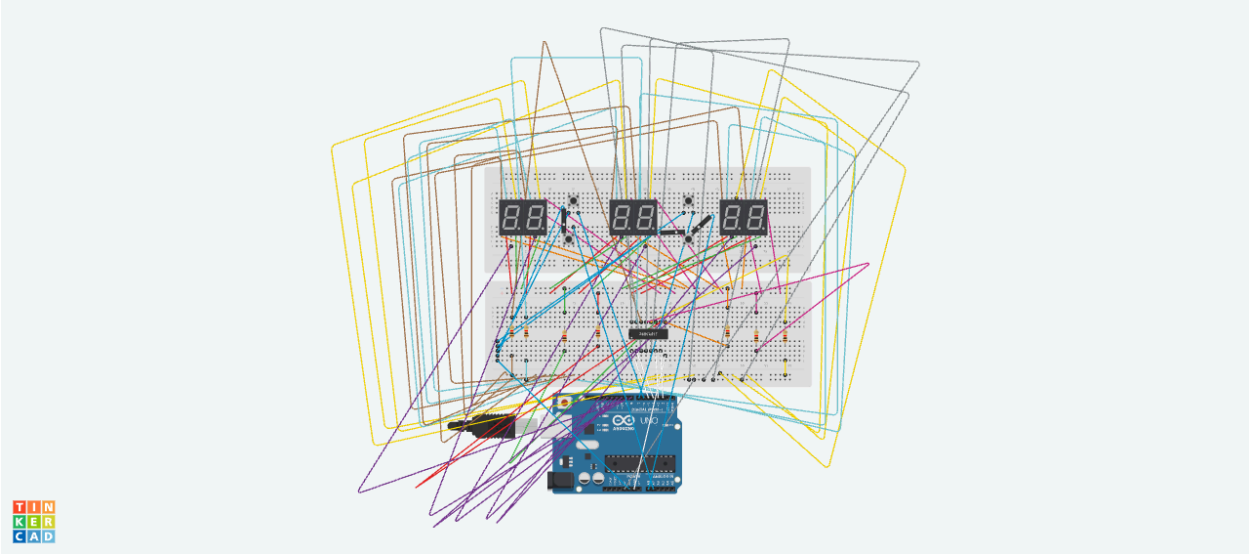


5. Push buttons

Four push buttons are connected to the circuit, with two dedicated to adjusting the hours and two for adjusting the minutes. For the hours, one button is used to increase the hour and the other to decrease it. Similarly,

the two minute buttons allow the user to increase or decrease the minutes, providing easy manual adjustment of the clock.
Ex: if the time is 12:00:00, using one push button for hours changes it to 1:00:00 while the other changes it to 11:00:00

Connections:
Hours up-arduino pin-6
Mins up-arduino pin-7
Hours down-arduino pin-A1
Mins down-arduino pin-A0



CODE USED

```
// ========================================================== //
// 4-DIGIT/6-DIGIT DIGITAL CLOCK USING 4x4 K-MAP TO 7447 LOGIC  //
// Common-Anode 7-segment displays via 7447 Decoder           //
// ========================================================== //


// BCD OUTPUTS TO 7447
```

```
const int bcdA = 2;   // Out_A = LSB
const int bcdB = 3;   // Out_B
const int bcdC = 4;   // Out_C
const int bcdD = 5;   // Out_D = MSB

// Push buttons
const int minUpBtnPin   = 6;
const int hourUpBtnPin  = 7;
const int minDownBtnPin = A1;
const int hourDownBtnPin = A0;

// Digit enable pins
const int digitHourTens  = 13;
const int digitHourUnits = 12;
const int digitMinTens   = 11;
const int digitMinUnits  = 10;
const int digitSecTens   = 9;
const int digitSecUnits  = 8;

// Time variables
int hours = 12;
int minutes = 0;
int seconds = 0;
unsigned long previousMillis = 0;
const unsigned long interval = 1000;
```

```
// Button states
bool lastMinUpState = HIGH;
bool lastHourUpState = HIGH;
bool lastMinDownState = HIGH;
bool lastHourDownState = HIGH;

// ============================================================ //
// SETUP                                                        //
// ============================================================ //
void setup() {
  pinMode(bcdA, OUTPUT);
  pinMode(bcdB, OUTPUT);
  pinMode(bcdC, OUTPUT);
  pinMode(bcdD, OUTPUT);

  pinMode(digitHourTens, OUTPUT);
  pinMode(digitHourUnits, OUTPUT);
  pinMode(digitMinTens, OUTPUT);
  pinMode(digitMinUnits, OUTPUT);
  pinMode(digitSecTens, OUTPUT);
  pinMode(digitSecUnits, OUTPUT);

  pinMode(minUpBtnPin, INPUT_PULLUP);
  pinMode(hourUpBtnPin, INPUT_PULLUP);
  pinMode(minDownBtnPin, INPUT_PULLUP);
  pinMode(hourDownBtnPin, INPUT_PULLUP);
```

```
  allDigitsOff();
}

// ============================================================ //
// MAIN LOOP                                                    //
// ============================================================ //
void loop() {
  unsigned long currentMillis = millis();

  // -------- 1 SECOND CLOCK --------
  if (currentMillis - previousMillis >= interval) {
    previousMillis += interval;
    seconds++;
    if (seconds >= 60) {
      seconds = 0;
      minutes++;
      if (minutes >= 60) {
        minutes = 0;
        hours++;
        if (hours > 12) {
          hours = 1;
        }
      }
    }
  }
```

```
  // -------- BUTTONS --------
  checkButtons();

  // -------- DISPLAY --------
  updateDisplay();
}

// ============================================================ //
// BUTTON CONTROL (FIXED SCOPE BRACKETS)                        //
// ============================================================ //
void checkButtons() {
  bool currentMinUp   = digitalRead(minUpBtnPin);
  bool currentHourUp  = digitalRead(hourUpBtnPin);
  bool currentMinDown = digitalRead(minDownBtnPin);
  bool currentHourDown = digitalRead(hourDownBtnPin);
  bool changed = false;

  // -------- Minutes UP --------
  if (lastMinUpState == HIGH && currentMinUp == LOW) {
    minutes++;
    if (minutes >= 60) {
      minutes = 0;
    }
    seconds = 0;
    changed = true;
```

```
}

// -------- Minutes DOWN --------
if (lastMinDownState == HIGH && currentMinDown == LOW) {
  minutes--;
  if (minutes < 0) {
    minutes = 59;
  }
  seconds = 0;
  changed = true;
}

// -------- Hours UP --------
if (lastHourUpState == HIGH && currentHourUp == LOW) {
  hours++;
  if (hours > 12) {
    hours = 1;
  }
  seconds = 0;
  changed = true;
}

// -------- Hours DOWN --------
if (lastHourDownState == HIGH && currentHourDown == LOW) {
  hours--;
  if (hours < 1) {
```

```
      hours = 12;
    }
    seconds = 0;
    changed = true;
  }

  // Save the states for the next loop run
  lastMinUpState = currentMinUp;
  lastHourUpState = currentHourUp;
  lastMinDownState = currentMinDown;
  lastHourDownState = currentHourDown;

  // Only debounce if an action actually happened
  if (changed) {
    delay(150);
  }
}

// ============================================================ //
// DISPLAY MULTIPLEXING                                         //
// ============================================================ //
void updateDisplay() {
  int hTens  = hours / 10;
  int hUnits = hours % 10;
  int mTens  = minutes / 10;
  int mUnits = minutes % 10;
```

```
  int sTens  = seconds / 10;
  int sUnits = seconds % 10;

  showDigit(hTens, digitHourTens);
  showDigit(hUnits, digitHourUnits);
  showDigit(mTens, digitMinTens);
  showDigit(mUnits, digitMinUnits);
  showDigit(sTens, digitSecTens);
  showDigit(sUnits, digitSecUnits);
}

// ============================================================ //
// 4-INPUT / 4-OUTPUT K-MAP BCD LOGIC                          //
// ============================================================ //
void showDigit(int n, int digitPin) {
  // 1. Clear old data by turning off ALL digits first (prevents ghosting)
  allDigitsOff();

  // 2. 4 Inputs extracted from the number (0-9)
  bool A = n & 1; // LSB (2^0)
  bool B = n & 2; //     (2^1)
  bool C = n & 4; //     (2^2)
  bool D = n & 8; // MSB (2^3)

  // 3. K-Map Minimised Equations for BCD-to-BCD conversion (0-9)
  // These equations evaluate standard BCD passing through to the 7447.
```

```
  bool outA = A;
  bool outB = B && !D;
  bool outC = C && !D;
  bool outD = D;

  // 4. Send the 4 K-Map processed outputs to the 7447 decoder
  digitalWrite(bcdA, outA);
  digitalWrite(bcdB, outB);
  digitalWrite(bcdC, outC);
  digitalWrite(bcdD, outD);

  // 5. Turn ON the specific target digit display
  digitalWrite(digitPin, HIGH);

  // 6. Multiplexing view delay
  delay(2);
}

// ============================================================ //
// TURN OFF ALL DIGITS                                          //
// ============================================================ //
void allDigitsOff() {
  digitalWrite(digitHourTens, LOW);
  digitalWrite(digitHourUnits, LOW);
  digitalWrite(digitMinTens, LOW);
  digitalWrite(digitMinUnits, LOW);
```

```
  digitalWrite(digitSecTens, LOW);
  digitalWrite(digitSecUnits, LOW);
}
```

TROUBLESHOOTING

1.Breadboard Issue:

At first, many of the displays were not lighting up, so we repeatedly tried changing the connections on the breadboard. Eventually, we had to use a second breadboard to get everything working. That is how we started the project with two breadboards. It was quite exhausting to rebuild and troubleshoot the circuit again and again, but it helped us remember the connections better and move forward with the project.

2.Wrong connections

Due to the small size of the breadboard holes and the limited spacing between them, making the connections was quite challenging. The large number of wires on the breadboard made it even more difficult to establish new connections. As a result, we made several incorrect connections to the 7447 IC. Due to the incorrect connections, several of the displays did not function properly. Some displays failed to turn on, while others showed random or incorrect patterns instead of displaying the intended numbers. which required us to repeatedly check and verify the wiring.

3.Improvement in Resistor Connection for Seven-Segment Displays

In the previous setup, the resistor was connected to the COM (common) pin instead of being connected individually to each segment. As a result, the available current was insufficient to drive all the segments properly, causing the displays to appear dim. To resolve this issue, we modified the circuit by connecting a separate resistor to each segment. This segment-wise resistor configuration provides sufficient current to each segment, resulting in brighter and more clearly visible displays.